IT Setup Guide — Option A (On-Prem / Local Server)

A from-scratch, do-this-then-that runbook for standing up the resiDNA app on company-controlled infrastructure. This assumes zero prior setup — nothing installed, no server picked yet. If you want the why behind any step, or the Option B (Azure + SharePoint) path instead, see IT_Deployment_Guide.md — this document is the concrete execution of that guide's Option A, chosen as the go-forward path. IT_Handoff_Checklist.md is the packing list of what to receive before starting here.

Steps are written for Linux (systemd, most common for this kind of internal tool) with the Windows equivalent called out wherever it differs.

0. Before you start

- ☐ The code handoff package from IT_Handoff_Checklist.md section 1 — `app/`, `Scripts/`, `.streamlit/config.toml`, `tests/`.
- ☐ A server/VM on the internal network, not internet-facing, that you control.
- ☐ Python 3.10 or newer available on it (this app was built and tested against Python 3.10).
- ☐ A company-issued TLS certificate for whatever internal hostname you'll use (e.g. `residna.internal.yourcompany.c`) or access to request one.
- ☐ Confirmation from compliance on the audit log retention period (step 6 below needs a number of months/years).
- ☐ If adding SSO (step 8): access to your Microsoft Entra ID tenant to register an application, or your Identity/IAM team's help doing so.

1. Get the code onto the server

Copy the handoff package to a directory you'll run the app from — this guide uses `/opt/residna_workflows` (Linux) / `C:\residna_workflows` (Windows). Keep `app/` and `Scripts/` as siblings; `app.py` locates `Scripts/` by a path relative to its own location and breaks on import if the layout changes.

```
sudo mkdir -p /opt/residna_workflows
sudo chown $(whoami) /opt/residna_workflows
# copy/unzip the handoff package into /opt/residna_workflows here
cd /opt/residna_workflows
ls app Scripts .streamlit/config.toml   # sanity check the layout survived the copy
```

2. Set up the Python environment

```
cd /opt/residna_workflows
python3 -m venv .venv
.venv/bin/pip install -r app/requirements.txt
```

Windows (PowerShell):

```
cd C:\residna_workflows
python -m venv .venv
.venv\Scripts\pip install -r app\requirements.txt
```

`app/requirements.txt` is pinned to exact tested versions — this should be a clean, reproducible install. Run it through your standard vulnerability scanning now, before going further.

3. Smoke-test it manually, unwrapped

Before wrapping this in a service, confirm it actually runs on this machine:

```
.venv/bin/streamlit run app/app.py --server.port 8501 --server.address 127.0.0.1
```

Open `http://127.0.0.1:8501` in a browser on the server itself (or via SSH port-forward: `ssh -L 8501:127.0.0.1:8501 user@server`) and upload a test QuantStudio file. Confirm results render and the PDF download works. `Ctrl+C` to stop once confirmed — this was only a manual smoke test, not the real running service.

If this step fails, stop here and fix it before proceeding — every step after this assumes a working app.

4. Wrap it as a persistent service

Today, closing the terminal (or a reboot) stops the app. Fix that first, before putting anything in front of it.

Linux — systemd. Create `/etc/systemd/system/residna.service`:

```
[Unit]
Description=resiDNA Streamlit App
After=network.target

[Service]
Type=simple
User=residna
WorkingDirectory=/opt/residna_workflows
Environment="RESIDNA_AUDIT_LOG=1"
ExecStart=/opt/residna_workflows/.venv/bin/streamlit run app/app.py --server.port
 ↪  8501 --server.address 127.0.0.1 --server.headless true
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Create a dedicated `residna` service account first (`sudo useradd -r -s /usr/sbin/nologin residna`) rather than running it as root or your own account, and make sure it owns `/opt/residna_workflows` (`sudo chown -R residna /opt/residna_workflows`) — it needs write access to create `Logs/` (step 6).

```
sudo systemctl daemon-reload
sudo systemctl enable --now residna
sudo systemctl status residna         # should show "active (running)"
curl http://127.0.0.1:8501            # should return HTML, not a connection error
```

Windows — NSSM (or a Windows container / Task Scheduler running at startup, if you prefer):

```
nssm install resiDNA "C:\residna_workflows\.venv\Scripts\streamlit.exe" "run
 ↪  app\app.py --server.port 8501 --server.address 127.0.0.1 --server.headless
 ↪  true"
nssm set resiDNA AppDirectory "C:\residna_workflows"
nssm set resiDNA AppEnvironmentExtra RESIDNA_AUDIT_LOG=1
nssm start resiDNA
```

`server.address 127.0.0.1` deliberately binds only to localhost — the reverse proxy in step 7 is what actually faces users; the raw Streamlit process should never be reachable directly.

5. Confirm **`RESIDNA_AUDIT_LOG=1`** took effect

The service file above already sets it, but verify before moving on — this is easy to silently get wrong (typo the variable name, forget to restart the service after editing the unit file, etc.):

```
sudo systemctl show residna -p Environment
```

Should include `RESIDNA_AUDIT_LOG=1`. Upload a test file through the running app and click Download PDF Report, then confirm a record landed:

```
tail -1 /opt/residna_workflows/Logs/audit.log
```

You should see one JSON line with a timestamp, the uploaded filename, and a `report_generated` event — no analytical results (DNA quantities, pass/fail per sample) in it, by design; see `app/audit_log.py`'s docstring for exactly what is and isn't captured.

6. Set up audit log retention

`Logs/audit.log` is a plain append-only file — nothing rotates or expires it on its own. Set that up now, using whatever retention period compliance specified (step 0):

Linux — logrotate. Create `/etc/logrotate.d/residna`:

```
/opt/residna_workflows/Logs/audit.log {
    monthly
    rotate 24
    compress
    missingok
    notifempty
    copytruncate
}
```

`rotate 24` = keep 24 months; change to match your compliance-specified retention. `copytruncate` is used deliberately — the app holds this file open in append mode and has no signal handler to reopen it, so a plain rotate/move would leave it writing to a now-unlinked file. `copytruncate` avoids that without needing to touch the app or restart the service on every rotation.

Windows has no logrotate equivalent built in — either a scheduled task that archives/truncates `Logs\audit.log` on the same schedule, or point `Logs/` at a network share with its own retention policy already configured.

Also make sure `Logs/audit.log` is included in whatever backup routine covers this server — it's the audit trail; losing it defeats the point.

7. Put a reverse proxy + TLS in front

Linux — Nginx. Install it (`sudo apt install nginx` / `sudo yum install nginx`), then create `/etc/nginx/sites-available/residna` (symlink into `sites-enabled`, or the equivalent on your distro):

```
server {
    listen 443 ssl;
    server_name residna.internal.yourcompany.com;

    ssl_certificate     /etc/ssl/certs/residna.crt;
    ssl_certificate_key /etc/ssl/private/residna.key;

    location / {
        proxy_pass http://127.0.0.1:8501;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

server {
    listen 80;
    server_name residna.internal.yourcompany.com;
    return 301 https://$host$request_uri;
}
```

The `Upgrade`/`Connection "upgrade"` headers are not optional — Streamlit uses a WebSocket for live updates, and the page loads but silently never updates without them. This is the single most common thing people miss putting Streamlit behind a proxy.

```
sudo nginx -t              # validate config syntax before reloading
sudo systemctl reload nginx
```

Windows — IIS, or your company's existing load balancer if you have one, following the same shape: TLS termination, proxy to `127.0.0.1:8501`, WebSocket support enabled (IIS: install the Application Request Routing + WebSocket Protocol modules).

Once this is up, `https://residna.internal.yourcompany.com` should load the app — port 8501 itself should no longer be reachable from anywhere except the server itself.

8. Add authentication

The app has no login built in today — anyone who can reach the URL can use it. Two options:

Option 1 — **`st.login()`** with Entra ID (recommended, matches existing company SSO). This needs two things:

1. An Entra ID App Registration (your Identity/IAM team, if you don't have tenant admin access): a redirect URI of `https://residna.internal.yourcompany.com/oauth2callback`, and a client secret.
2. `.streamlit/secrets.toml` on the server (not in the code handoff package — create this fresh, and keep it out of any copy of these files that reaches version control):

   ```
   [auth]
   redirect_uri = "https://residna.internal.yourcompany.com/oauth2callback"
   cookie_secret = "<generate a long random string>"
   client_id = "<from the App Registration>"
   client_secret = "<from the App Registration>"
   server_metadata_url = "https://login.microsoftonline.com/<your-tenant-
    ↪  id>/v2.0/.well-known/openid-configuration"
   ```

3. One small code change to `app/app.py`, near the top (after `st.set_page_config`, before anything else renders):

   ```
   if not st.user.is_logged_in:
       st.login()
       st.stop()
   ```

   This is the one piece of "code" in this otherwise infrastructure-only guide — it's two lines, and it's what makes `st.user` (already read by `app/audit_log.py`, for the `authenticated_user` field) start returning a real identity instead of `None`. No other code changes needed.

Restart the service after adding secrets.toml and the code change:

```
sudo systemctl restart residna
```

Option 2 — an auth proxy (e.g. oauth2-proxy) sitting between Nginx and Streamlit, if you'd rather keep auth entirely outside the app. In that case skip the code change above; `authenticated_user` in the audit log stays `None` (submitter name, self-reported in the app's Signatures section, remains the only identity signal) unless you also forward the proxy's authenticated identity as a header the app reads — a further customization not covered here.

9. Restrict network reachability

- Bind Nginx/IIS to the internal DNS name/IP only — confirm nothing routes to this server from outside the corporate network/VPN.
- Firewall rule: allow 443 (and 80, for the redirect) from internal ranges only; nothing else inbound.
- Confirm port 8501 (the raw Streamlit process) is not reachable from any other machine — only from `127.0.0.1` on the server itself.

10. Go-live verification checklist

- ☐ `https://residna.internal.yourcompany.com` loads over HTTPS with a valid company certificate (no browser warning).
- ☐ Uploading a QuantStudio file and generating a PDF report works end-to-end.
- ☐ Login is required before the app is usable (step 8) — confirm by trying the URL in a private/incognito window.
- ☐ `Logs/audit.log` gets a new line after each PDF download (step 5).
- ☐ Rebooting the server brings the app back up on its own (`sudo reboot`, then re-check the URL after it comes back) — confirms step 4's persistent service is actually persistent.
- ☐ `sudo systemctl status residna` shows `Restart=always` behavior: `sudo systemctl kill residna` then re-check — it should come back within a few seconds on its own.
- ☐ `python -m pytest tests -v` (from `/opt/residna_workflows`, with `.venv` active and `tests/requirements.txt` installed) passes, confirming the app behaves correctly in this environment. `test_app_golden.py` will skip if you didn't also copy real sample files into `Data/` — that's expected, not a failure (see README.md).

11. Ongoing maintenance

- Dependency updates: `app/requirements.txt` is pinned deliberately. Don't casually `pip install --upgrade`; re-test against `tests/` first, then update the pin.
- Log rotation: confirm step 6's logrotate/scheduled task is actually firing (`sudo logrotate -d /etc/logrotate.d/residna` to dry-run it) a month or two after go-live, not just configured.
- Certificate renewal: whatever your company's normal TLS renewal process is applies here too — this is a company-issued cert like any other internal service's.
- Restarting after **`secrets.toml`**/**`.streamlit/config.toml`** changes: Streamlit doesn't hot-reload these — `sudo systemctl restart residna` after any change to either.